

dataQ 자동 표준화 추천 시스템 첫 설계

시점: 2026-03-27

초점의 이동

앞선 일지(01)에서는 "진단 이슈를 한 화면에서 해결한다"는 문제를 풀고 있었다. 이 시점에 초점이 한 번 더 옮겨갔다. 원인은 개발자 입장에서 들어본 이야기였다.

"테이블에 컬럼 몇 개 추가하려는데 이게 표준에 맞는지 모르겠다. 표준 담당자에게 물어봐야 하는데 한 건에 3~5분 걸리고, 100개면 하루가 다 간다."

진단은 이미 만들어진 테이블에서 문제를 찾는다. 그런데 개발자가 필요로 하는 건 그 반대다. 테이블을 만들기 전에 한글 컬럼명을 가지고 가서 "이걸 표준으로 정리하면 어떻게 되냐"고 물어보는 것. 질문의 시점이 다르기 때문에 비도 달라야 한다. 그래서 "표준화 추천"이라는 이름으로 별도 메뉴를 만들기로 했다.

목표 워크플로우

개발자가 엑셀(또는 텍스트)로 한글명만 던지면, 시스템이 다음을 자동으로 뽑아서 돌려준다.

- 이 한글명이 이미 등록된 용어인지 아닌지
- 아니면 어떤 단어들로 구성되는지 (사용자 + 코드 + 명)
- 각 단어의 표준 영문약어는 무엇인지 (USR + CD + NM)
- 전체 영문 표준명은 어떻게 조합되는지 (USR_CD_NM)
- 어떤 도메인에 붙는 게 적절한지 (마지막 단어가 "명"이면 명칭 도메인 후보를 우선)

개발자가 리뷰 테이블에서 결과를 훑고, 문제없는 건 승인, 미등록 단어가 있는 건 그 단어 정보만 채우고 승인. 최종적으로 단어-용어 일괄 등록 + 선택된 DBMS용 DDL 스크립트 출력까지.

100건을 5분 안에 끝내는 것이 목표였다.

분석 파이프라인

한글명 하나를 받아서 결과를 내기까지의 단계를 이렇게 쪼갰다.

한글명 "사용자코드명"

- ① 기존 용어 완전매칭 (TB_TERMS.TERMS_NM)
 - 있으면 그대로 "기등록" 상태로 반환, 나머지 단계 건너뛰
- ② OKT 형태소 분석
 - [사용자, 코드, 명]
- ③ 단어 매칭 (TB_WORD)
 - 각 토큰을 TB_WORD에서 찾음, 후보 복수면 리스트로
- ④ 최적 단어 선택 (후보 중 하나)
 - 우선순위: 사용 빈도 > 영문약어 길이 > 공통표준여부
- ⑤ 영문약어 조합
 - USER + CD + NM → "USER_CD_NM"
- ⑥ 도메인 자동 추천
 - 마지막 단어("명")의 도메인분류 → 해당 분류의 도메인 목록
 - 1개면 자동 선택, 복수면 사용 빈도순
- ⑦ 상태 판정
 - REGISTERED / AUTO / PARTIAL / FAILED 중 하나

네 가지 최종 상태는 이런 의미였다.

- REGISTERED: 이미 등록된 용어. 추천할 게 없음. 참조만 하면 됨.
- AUTO: 모든 단어가 TB_WORD에 있고 도메인 추천도 성공. 개발자는 "승인"만 누르면 됨.
- PARTIAL: 일부 단어가 미등록. 개발자가 미등록 단어 정보를 채워야 함.
- FAILED: 형태소 분석 실패 또는 대부분이 미등록. 사실상 수동 등록.

단어 선택의 우선순위

한 토큰에 대해 TB_WORD 후보가 여러 개 있을 수 있다 (사용자 → USER, USAGER). 이 중 하나를 자동 선택해야 한다. 처음에 잡은 점수 체계는 이랬다.

- 사용 빈도 × 100 (TB_TERMS_WORDS에 등장한 횟수)
- (10 - 영문약어 길이) × 10 (짧을수록 가산 — 관례상 짧은 약어가 표준으로 본다)
- 공통표준여부가 Y면 +50

이 중 "짧을수록 가산"이라는 부분은 나중에 다시 논의 거리가 된다. 짧은 약어가 항상 좋은 건 아니라서 방향을 뒤집을지 한 번 검토하기로 기록만 해뒀다.

미등록 단어에 대한 추천값

PARTIAL 상태일 때 사용자가 미등록 단어의 정보를 손으로 채우면 부담이 크다. 그래서 미등록 단어에도 "추천값"을 붙이기로 했다. 추천값은 어디서 나오나.

- 기존 TB_WORD의 패턴 분석 — 2글자 한글 단어는 보통 2~4자 영문약어라는 정도의 통계
- 마지막 글자로 도메인분류 추정 — "금"으로 끝나면 금액, "일/자"로 끝나면 날짜, "명"으로 끝나면 명칭, 그 외는 기타
- 영문명은 구글 번역 같은 걸 쓰기엔 일관성이 안 나와서, 일단 이 시점에는 수동 입력을 유도하고 추천값은 약어와 도메인분류만 제시

이후 단계에서 TB_WORD_DICT(추천 사전)라는 별도 테이블을 도입한다. 이건 TB_WORD와는 다르다. TB_WORD는 "표준 단어"고 추가는 까다롭다. TB_WORD_DICT는 "자주 등장하는 일반 단어와 그 관례적 영문약어"를 미리 채워둔 것이다. 개발자가 "핸드폰" 같은 단어를 넣으면 TB_WORD에는 없지만 TB_WORD_DICT에는 HNDPHN / HANDPHONE 같은 게 있어서 추천 품질이 올라간다. 이 시드 데이터는 처음엔 337건으로 시작했다가 나중에 행안부 행정용어·국가기록원 기록관리 단어까지 붙이면서 48,159건까지 늘어난다.

API 설계의 골격

분석과 등록을 두 API로 나눴다.

- `POST /api/std/analyzeTermsBatch` — 한글명 리스트를 받아서 분석 결과 리스트를 반환. 읽기 전용. 호출 한 번에 여러 건을 처리. 호출 시작할 때 TB_WORD, TB_TERMS, TB_DOMAIN을 통째로 캐시로 로드하고, 이 캐시 위에서 각 한글명을 처리. 수천 건 규모면 DB 왕복 없이 거의 메모리 위에서 끝난다.
- `POST /api/std/registerTermsBatch` — 사용자가 리뷰한 결과를 받아서 TB_WORD INSERT → TB_TERMS INSERT 순서로 등록. newWords가 있으면 선등록, wordId가 null이면 newWords 인덱스로 매핑.

등록 API는 "단어 먼저, 그 다음 용어"라는 순서 의존성이 있다. 트랜잭션 하나 안에서 처리해야 한다. 이 부분에서 나중에 FK 참조 에러가 몇 번 터진다.

성능 관련 초기 판단

OKT는 초기 로딩이 느리다. 서버 시작 시 한 번 워밍업해서 첫 요청 지연을 피한다. 단어/용어/도메인은 분석 시작 시 전체 캐시. 대량 입력(1000건+)이 오면 50건씩 청크 처리하면서 WebSocket으로 진행률을 보낸다. 이건 동기 처리가 아니라 q-executor 쪽에서 돌릴 가능성도 열어뒀다.

이 시점에 알지 못했던 것

돌이켜 보면 이 설계는 OKT 형태소 분석 결과를 상당히 신뢰하고 있었다. "OKT가 [사용자, 코드, 명]을 뽑아준다"는 전제 위에서 단어 매칭을 설계했다. 실제 데이터를 돌리기 전까지는 OKT가 얼마나 "이상하게" 나올 수 있는지 감이 없었다. 특히 복합명사에 대한 OKT의 토큰나이징이 단어 사전 기반 분리에 어떤 영향을 주는지는 이 시점에 고민하지 않았다.

다음 일지(03)에서 첫 구현을 테스트로 돌리면서 이 문제가 서서히 드러나고, 일지 04에서는 단어 분리 알고리즘을 통째로 갈아엎게 된다.